Lab 08

IPv6 and Neighbor Discovery on Linux

Communication Technologies for the Internet of Things | Lecture 08

Aim and context

Lecture 08 introduces IPv6 addressing, stateless autoconfiguration and the Neighbor Discovery Protocol, in contrast with IPv4 and NAT. This lab makes those ideas observable by building a small network with Linux network namespaces and capturing real traffic.

The work takes place in an Ubuntu virtual machine, created and deleted with a single command, and does not require access to the course server. It is meant to be done during the class, at each student's pace, and is **not an assignment to hand in**.

> **Note**
>
> Network namespaces are the same isolation mechanism that underlies containers. Each has its own IPv6 stack, with interfaces, addresses, routes and a neighbour table, so it behaves like a separate machine. The same sequence of commands works on a Raspberry Pi running Linux.

Learning outcomes

By the end of this lab, the student should be able to:

- Identify the link-local address of an interface and relate it to its MAC address (EUI-64).
- Observe the exchange of Neighbor Solicitation and Neighbor Advertisement messages and the neighbour table.
- Verify the solicited-node multicast address associated with an interface.
- Observe stateless autoconfiguration from a Router Advertisement.

- Demonstrate the effect of NAT in IPv4 and inspect its translation table.

Requirements

- A computer running Windows, macOS or Linux.
- About 1 GB of free memory for the virtual machine and 5 GB of disk.
- An Internet connection, needed to fetch the image and packages.
- Estimated duration: 60 to 90 minutes.

How to use this guide

Each step presents, in this order, the aim, the context, the theoretical background, the commands to run and the expected result. Read the whole step before running the first command. The boxes always have the same meaning:

Command to run

```
Command to run in the terminal.
```

Expected result (values may vary)

```
Result that should be displayed.
```

Analyse

A moment to pause and interpret the result, on your own or in discussion with colleagues.

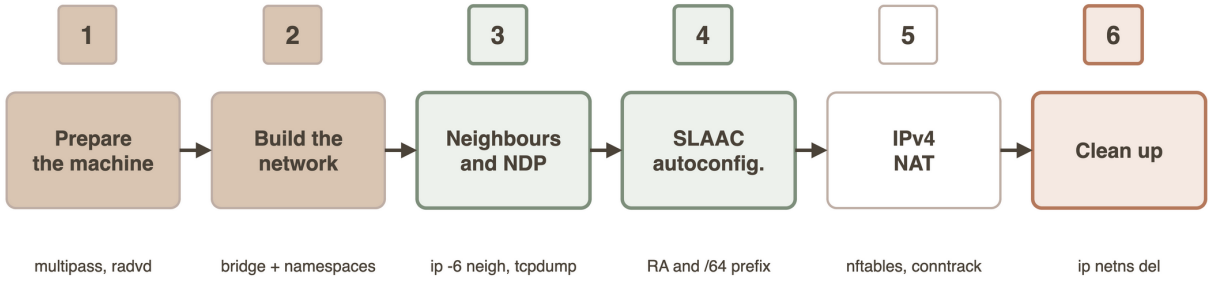
Possible error and fix

An error that may occur and how to fix it.

Commands can be copied and pasted. The results shown are examples: MAC addresses, link-local addresses and times differ from computer to computer. Whenever a value appears between angle brackets, such as `<address>`, replace it, brackets included, with the real value shown on screen.

Overview

The figure below summarises the sequence of the guide. Steps 3 and 4 are the core observation: the neighbour discovery protocol and stateless autoconfiguration, in real traffic.



Steps 3 and 4 show, in real traffic, what the lecture described: NDP and stateless autoconfiguration

How to read the figure

The six steps read from left to right. The first two, in beige, prepare the environment and build the network. Steps 3 and 4, in green, are the core: you observe how nodes discover each other and how they obtain addresses without a server. Step 5, in white, shows IPv4 NAT for contrast. Step 6, in a reddish tone, closes the work. Below each box, a caption names the main tool of the step.

Step 1 — Preparing the machine

Aim. Create an Ubuntu virtual machine and install the required network tools.

Context. `multipass` creates Ubuntu virtual machines with a single command and works on Windows, macOS and Linux. The tools `radvd`, `tcpdump` and `conntrack` are used in the following steps.

Theoretical background

A virtual machine isolates the lab from the student's system: the namespaces, bridges and firewall rules created exist only inside it and disappear when it is deleted. `radvd` is a program that announces IPv6 prefixes through Router Advertisement messages, and plays in this lab the role of the router described on the autoconfiguration slide. `tcpdump` shows the packets crossing an interface, and `conntrack` lists the NAT translation table.

To analyse in this step: only the preparation. Nothing that is installed interferes with the computer's network.

1.1 Creating the virtual machine

On macOS, install with `brew install --cask multipass`. On Windows, use the installer from https://multipass.run. On Linux, `sudo snap install multipass`. Then:

Command to run

```
multipass launch 22.04 --name tciot-ipv6 --cpus 2 --memory 1G --disk 5G
multipass shell tciot-ipv6
```

Expected result (values may vary)

```
Launched: tciot-ipv6
ubuntu@tciot-ipv6:~$
```

From this point on, all commands are run inside the virtual machine.

1.2 Installing the tools

Command to run

```
sudo apt update
sudo apt install -y radvd tcpdump conntrack nftables curl
```

Possible error and fix

launch failed or a stalled execution: the computer does not have enough free memory. Close other applications and try again. If the machine already exists, run `multipass delete tciot-ipv6`, `multipass purge` and repeat.

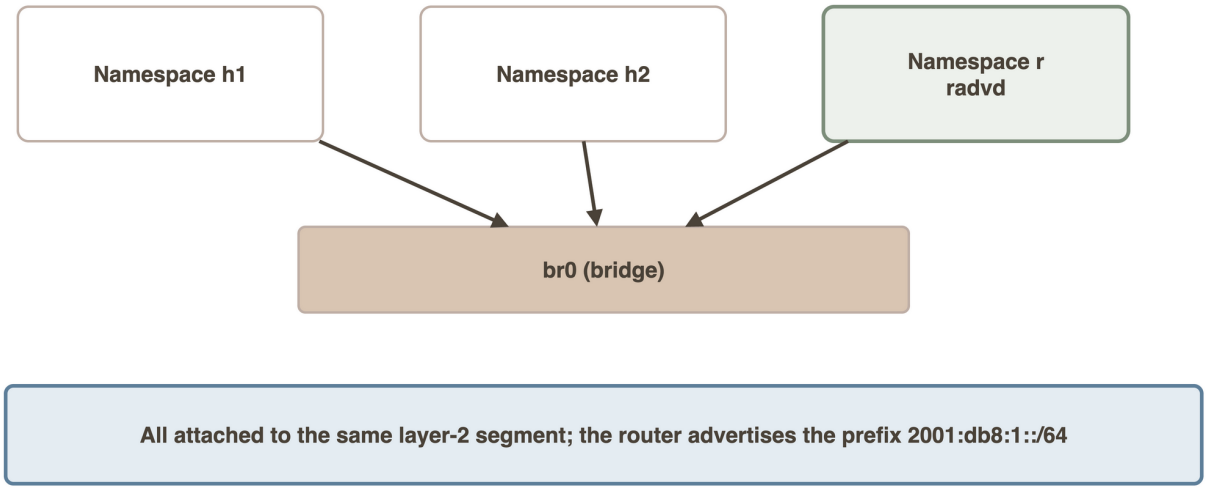
Check before moving on

The command `tcpdump --version` shows the installed version.

Step 2 — Building the network

Aim. Create three logical machines, `h1`, `h2` and `r`, attached to the same layer-2 segment.

Context. Each network namespace behaves like a machine. A Linux bridge, `br0`, plays the role of a switch that connects them.



How to read the figure

The three namespaces are at the top and the bridge, in beige, below. Each arrow represents a virtual cable between the namespace's `eth0` interface and the bridge. `r` is green because it will be the router, and `h1` and `h2` are end hosts. The bottom box gives the prefix the router will announce in step 4.

Theoretical background

A *veth* pair is a virtual cable: what enters one end leaves the other. One end stays in the namespace, named `eth0`, and the other is attached to the bridge. When an interface is brought up, the kernel immediately gives it a link-local address, with prefix `fe80::/10`, and runs duplicate address detection. So, after a second or two, each machine already has an IPv6 address with no configuration at all.

To analyse in this step: what exists before any manual configuration. No address was assigned, and the interfaces nevertheless end up with an IPv6 address.

2.1 Creating the bridge and the namespaces

Command to run

```
sudo ip link add br0 type bridge
sudo ip link set br0 type bridge mcast_snooping 0
sudo ip link set br0 up
for n in h1 h2 r; do
  sudo ip netns add $n
  sudo ip link add v-$n type veth peer name eth0 netns $n
  sudo ip link set v-$n master br0
  sudo ip link set v-$n up
  sudo ip netns exec $n ip link set lo up
  sudo ip netns exec $n ip link set eth0 up
done
```

2.2 Verification

Command to run

```
sudo ip netns list
sudo ip netns exec h1 ip -6 addr show dev eth0
```

Expected result (values may vary)

```
r
h2
h1
2: eth0@if7: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 state UP
    inet6 fe80::c4a:1eff:fe3c:4d5e/64 scope link
       valid_lft forever preferred_lft forever
```

Possible error and fix

Cannot find device or `File exists`: a previous command failed or was repeated. Remove what was left half-done with `sudo ip netns del h1` (and `h2`, `r`) and `sudo ip link del br0`, and repeat the step.

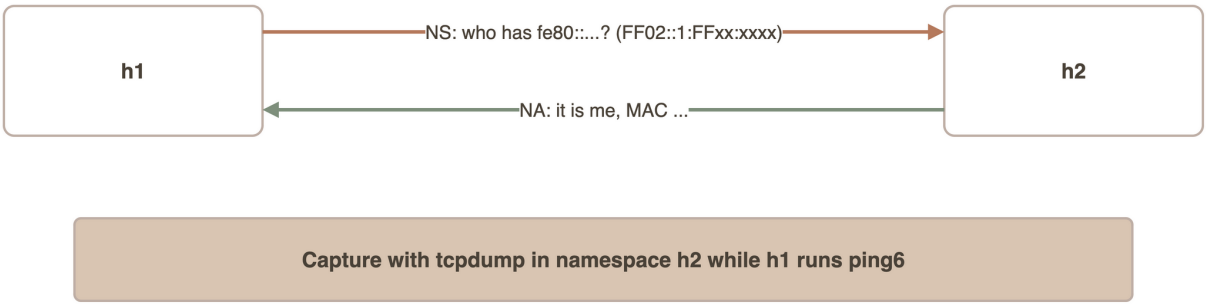
Check before moving on

There are three namespaces and each `eth0` has an address starting with `fe80::`.

Step 3 — Neighbours and Neighbor Discovery

Aim. Observe the link-local address, the neighbour table and the exchange of Neighbor Solicitation and Neighbor Advertisement messages.

Context. In IPv6 there is no ARP. The same job is done by the Neighbor Discovery Protocol, with ICMPv6 messages sent to solicited-node multicast addresses.



How to read the figure

`h1` wants to send a packet to `h2`, but does not yet know its physical address. The upper arrow is the Neighbor Solicitation, sent to `h2`'s solicited-node multicast address. The lower arrow is the Neighbor Advertisement, in unicast, which carries the physical address. The bottom box says where the capture is made: on `h2`, while `h1` sends the ping.

Theoretical background

In classic autoconfiguration, the interface identifier is derived from the MAC address: FFFE is inserted in the middle and the universal/local bit is flipped. That is why the link-local address of an interface with MAC `aa:bb:cc:dd:ee:ff` contains `a8bb:ccff:fedd:eeff`. Recent kernels may use opaque identifiers, in which case the relation is not visible.

To resolve a neighbour's physical address, the node sends a Neighbor Solicitation to the solicited-node multicast address, formed by `ff02::1:ff` and the last 24 bits of the address being looked up. Only nodes whose address ends in the same 24 bits process the message. The result is stored in the neighbour table, where each entry has a state such as `REACHABLE` or `STALE`.

To analyse in this step: the relation between MAC and address, and the fact that the solicitation is not broadcast to the whole network.

3.1 Addresses and the relation to the MAC

Command to run

```
sudo ip netns exec h1 ip link show eth0 | grep ether
sudo ip netns exec h1 ip -6 addr show dev eth0 scope link
sudo ip netns exec h2 ip -6 addr show dev eth0 scope link
```

Analyse

Compare the MAC address of `h1` with the last 64 bits of its link-local address. Which steps take you from one to the other? What change did the first octet undergo?

3.2 Ping to all nodes

Command to run

```
sudo ip netns exec h1 ping -6 -c 2 -I eth0 ff02::1
```

Expected result (values may vary)

```
64 bytes from fe80::c4a:1eff:fe3c:4d5e%eth0: icmp_seq=1 ttl=64 time=0.07 ms
64 bytes from fe80::7c2d:9ff:fe11:2233%eth0: icmp_seq=1 ttl=64 time=0.31 ms (DUP!)
64 bytes from fe80::a4b8:5cff:fe90:1a77%eth0: icmp_seq=1 ttl=64 time=0.33 ms (DUP!)
```

The address `ff02::1` designates all nodes on the link. `h1` itself, `h2` and `r` replied. The `DUP!` warnings are expected: a single message received several replies.

3.3 Capturing the discovery

Get the link-local address of `h2` and clear `h1`'s neighbour table:

Command to run

```
sudo ip netns exec h2 ip -6 addr show dev eth0 scope link | grep inet6
sudo ip netns exec h1 ip -6 neigh flush dev eth0
```

In a second terminal of the virtual machine, start the capture on `h2`:

Command to run

```
sudo ip netns exec h2 tcpdump -ni eth0 -c 4 icmp6
```

In the first terminal, send a request to `h2`, replacing the address:

Command to run

```
sudo ip netns exec h1 ping -6 -c 1 -I eth0 <address-of-h2>
```

Expected result (values may vary)

```
IP6 fe80::c4a:1eff:fe3c:4d5e > ff02::1:ff11:2233: ICMP6, neighbor solicitation, who has
    fe80::7c2d:9ff:fe11:2233
IP6 fe80::7c2d:9ff:fe11:2233 > fe80::c4a:1eff:fe3c:4d5e: ICMP6, neighbor advertisement, tgt
     is fe80::7c2d:9ff:fe11:2233
IP6 fe80::c4a:1eff:fe3c:4d5e > fe80::7c2d:9ff:fe11:2233: ICMP6, echo request, id 17, seq 1
IP6 fe80::7c2d:9ff:fe11:2233 > fe80::c4a:1eff:fe3c:4d5e: ICMP6, echo reply, id 17, seq 1
```

Analyse

What is the destination address of the first message? Compare its last 24 bits with those of `h2`'s address. Which message plays, in IPv6, the role of the ARP request?

3.4 Neighbour table and multicast groups

Command to run

```
sudo ip netns exec h1 ip -6 neigh
sudo ip netns exec h2 ip maddr show dev eth0
```

Expected result (values may vary)

```
fe80::7c2d:9ff:fe11:2233 dev eth0 lladdr 7e:2d:09:11:22:33 REACHABLE
    inet6 ff02::1:ff11:2233
    inet6 ff02::1
    inet6 ff01::1
```

Possible error and fix

The capture does not show the Neighbor Solicitation: `h1`'s neighbour table already had the entry. Repeat `neigh flush` immediately before the ping.
Network is unreachable: the interface was not given with `-I eth0`. Link-local addresses are only valid on their own interface.

Check before moving on

You saw the Neighbor Solicitation and Neighbor Advertisement pair, and the `ff02::1:ff...` group appears in `h2`'s group list.

Step 4 — Stateless autoconfiguration

Aim. Make the router announce a prefix and observe the hosts building global addresses without a server.

Context. This is the most valuable IPv6 feature for IoT: a device connects and obtains an address, with no manual configuration and no DHCP.

Theoretical background

The router sends Router Advertisement messages periodically, and in response to a Router Solicitation, carrying the network prefix. If the *autonomous* option is on, each host concatenates the 64-bit prefix with its own interface identifier and obtains a global address, after running duplicate detection. The router keeps no record of the assignments, which is why autoconfiguration is called stateless. The same message also announces the router as the default gateway, and the host adds a route to it.

To analyse in this step: who knows what. The router knows only the prefix; each host computes its own address.

4.1 Router configuration

Create the file `radvd.conf` in the virtual machine's home folder, for example with `nano`:

Contents of file `radvd.conf`

```
interface eth0 {
    AdvSendAdvert on;
    MinRtrAdvInterval 3;
    MaxRtrAdvInterval 10;
    prefix 2001:db8:1::/64 {
        AdvOnLink on;
        AdvAutonomous on;
    };
};
```

The prefix `2001:db8::/32` is reserved for documentation, so it is safe for tests.

4.2 Running and checking

Command to run

```
sudo ip netns exec r sysctl -w net.ipv6.conf.all.forwarding=1
sudo ip netns exec r radvd -n -m stderr -C ~/radvd.conf
```

The terminal stays busy. In a second terminal, check `h1`'s addresses and route:

Command to run

```
sudo ip netns exec h1 ip -6 addr show dev eth0
sudo ip netns exec h1 ip -6 route
```

Expected result (values may vary)

```
inet6 2001:db8:1:0:c4a:1eff:fe3c:4d5e/64 scope global dynamic mngtmpaddr
inet6 fe80::c4a:1eff:fe3c:4d5e/64 scope link
default via fe80::a4b8:5cff:fe90:1a77 dev eth0 proto ra metric 1024 expires 1790sec
2001:db8:1::/64 dev eth0 proto kernel metric 256 expires 86390sec
```

Analyse

The global address uses the announced prefix and the same last 64 bits as the link-local address. Where was the prefix information before the announcement? Who chose the last 64 bits?

4.3 Communication with a global address

Command to run

```
sudo ip netns exec h2 ip -6 addr show dev eth0 | grep global
sudo ip netns exec h1 ping -6 -c 2 <global-address-of-h2>
```

Possible error and fix

The host does not get a global address: confirm that `radvd` is running and that `forwarding` is 1 on `r`. On `h1`, `sysctl net.ipv6.conf.eth0.accept_ra` should return 1. **radvd: config file permissions**: the file must not be writable by other users. Run `chmod 644 ~/radvd.conf`.

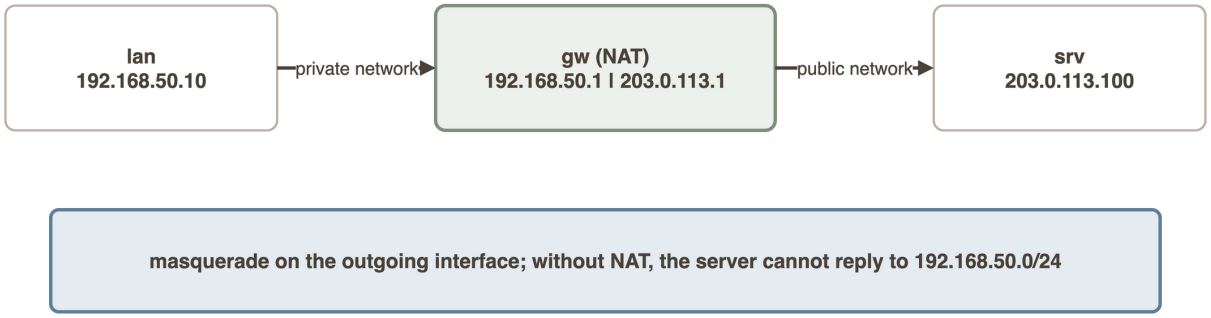
Check before moving on

`h1` and `h2` have a global address with the prefix `2001:db8:1::/64` and communicate through it.

Step 5 — IPv4 NAT, for contrast

Aim. Show that a machine with a private address can only talk to the outside through address translation, and inspect the NAT table.

Context. The lecture presented NAT as a patch for IPv4 address exhaustion. Here its effect is observed from both sides.



How to read the figure

On the left is the machine on the private network, `lan`. In the centre is `gw`, with two interfaces: one on the private network and one on the public network. On the right is the server `srv`, which only knows the public network. The bottom box summarises what will be shown: without translation, the server receives requests from an address it has no route to and cannot reply.

Theoretical background

The addresses in the 192.168.0.0/16 range are private and are not routed on the Internet. When a packet with such a source address reaches a public server, the server does not know how to reply. *Masquerade* replaces, on each outgoing packet, the source address with that of the router's outgoing interface and records the correspondence in *connection tracking*. On the reply, the record allows the inverse operation.

To analyse in this step: the address the server sees. It is not that of the machine that made the request.

5.1 Topology

First, remove the network from step 2 and stop `radvd` (`Ctrl+C` in the terminal where it runs):

Command to run

```
for n in h1 h2 r; do sudo ip netns del $n; done
sudo ip link del br0
```

Command to run

```
for n in lan gw srv; do sudo ip netns add $n; done
sudo ip link add l-lan type veth peer name l-gw
sudo ip link add w-gw type veth peer name w-srv
sudo ip link set l-lan netns lan
sudo ip link set l-gw netns gw
sudo ip link set w-gw netns gw
sudo ip link set w-srv netns srv
sudo ip netns exec lan ip addr add 192.168.50.10/24 dev l-lan
sudo ip netns exec gw ip addr add 192.168.50.1/24 dev l-gw
sudo ip netns exec gw ip addr add 203.0.113.1/24 dev w-gw
sudo ip netns exec srv ip addr add 203.0.113.100/24 dev w-srv
for p in "lan l-lan" "gw l-gw" "gw w-gw" "srv w-srv"; do
  set -- $p; sudo ip netns exec $1 ip link set $2 up
done
sudo ip netns exec lan ip route add default via 192.168.50.1
sudo ip netns exec gw sysctl -w net.ipv4.ip_forward=1
```

5.2 Request without NAT

Start an HTTP server on `srv`, in another terminal:

Command to run

```
sudo ip netns exec srv python3 -m http.server 8000
```

Command to run

```
sudo ip netns exec lan curl -s -m 5 -o /dev/null -w "%{http_code}\n" http
    ://203.0.113.100:8000/
```

Expected result (values may vary)

```
000
```

The request reaches the server, but the reply does not come back, because `srv` has no return route to the private network.

5.3 Request with NAT

Command to run

```
sudo ip netns exec gw nft add table ip nat
sudo ip netns exec gw nft add chain ip nat post '{ type nat hook postrouting
    priority 100 ; }'
sudo ip netns exec gw nft add rule ip nat post oifname "w-gw" masquerade
sudo ip netns exec lan curl -s -m 5 -o /dev/null -w "%{http_code}\n" http
    ://203.0.113.100:8000/
```

Expected result (values may vary)

```
200
```

In the server's terminal, the access log shows the source address:

Expected result (values may vary)

```
203.0.113.1 - - [14/Oct/2026 10:21:07] "GET / HTTP/1.1" 200 -
```

5.4 Translation table

Command to run

```
sudo ip netns exec gw conntrack -L
```

Expected result (values may vary)

```
tcp  6 118 TIME_WAIT src=192.168.50.10 dst=203.0.113.100 sport=45822 dport=8000
     src=203.0.113.100 dst=203.0.113.1 sport=8000 dport=45822 [ASSURED]
```

Analyse

The server recorded 203.0.113.1 as the source of the request, not 192.168.50.10. Where was the information kept that allows the reply to be returned to the right machine? What problem would arise if `srv` wanted to initiate a connection to `lan`?

Possible error and fix

curl returns 000 even with NAT: confirm that `ip_forward` is 1 on `gw` and that the HTTP server is still running. **conntrack: command not found**: install with `sudo apt install -y conntrack`.

Check before moving on

The request failed without NAT, worked with NAT, and the `conntrack` table shows both versions of the connection.

Step 6 — Cleaning up

Aim. Stop the services and delete the network and the virtual machine.

6.1 Inside the virtual machine

Command to run

```
sudo pkill -f "http.server 8000"
sudo pkill radvd
for n in lan gw srv h1 h2 r; do sudo ip netns del $n 2>/dev/null; done
sudo ip link del br0 2>/dev/null
exit
```

6.2 On the student's computer

Command to run

```
multipass delete tciot-ipv6
multipass purge
multipass list
```

Expected result (values may vary)

```
No instances found.
```

Note

To repeat the lab, simply repeat steps 1 to 5. The `radvd.conf` file and the commands are in this guide.

Troubleshooting: quick reference

Symptom	Fix
`multipass: command not found`	Close and reopen the terminal. If it persists, repeat the installation.
`File exists` or `Cannot find device`	Remove namespaces and bridge with `ip netns del` and `ip link del br0`, and repeat.
Address in `tentative` state	Wait one or two seconds for duplicate detection.
No reply to ping with `fe80::`	Give the interface: `-I eth0` or `%eth0`.
No Neighbor Solicitation in the capture	Run `ip -6 neigh flush dev eth0` before the ping.
No global address	Check `radvd` and `forwarding=1` on `r`.
`radvd: config file permissions`	`chmod 644 ~/radvd.conf`.
`curl` returns `000` with NAT	Check `ip_forward`, the `masquerade` rule and the HTTP server.

Commands used

Command	Purpose
`ip netns add / exec / del`	Create, use and delete a namespace
`ip link add ... type veth`	Create a virtual cable
`ip -6 addr show`	List IPv6 addresses
`ip -6 neigh`	Show the neighbour table
`ip maddr show`	List multicast groups
`ping -6 -I eth0 ff02::1`	Reach all nodes on the link
`tcpdump -ni eth0 icmp6`	Capture ICMPv6 messages
`radvd -C radvd.conf`	Announce a prefix
`nft add rule ... masquerade`	Enable outgoing NAT
`conntrack -L`	List the translation table

Review questions

The following questions are for discussion and need not be handed in.

1. How is the link-local address of an interface related to its MAC address?
2. What destination address did the Neighbor Solicitation use, and why was it not broadcast to the whole network?
3. Who decided the prefix and who decided the last 64 bits of the global address?
4. What information would the server lose without NAT, and what does `gw` keep in order to return the reply?
5. What advantage does stateless autoconfiguration have in an installation with hundreds of sensors?

Summary

The lab made it possible to observe, in real traffic, the central ideas of Lecture 08:

- Each IPv6 interface obtains a **link-local address** with no configuration, derived from the MAC.
- **Neighbor Discovery** replaces ARP, with solicitations sent to the solicited-node multicast address.
- **Stateless autoconfiguration** lets a host obtain a global address from the announced prefix, with no server.
- **NAT** lets private machines communicate with the outside, but keeps state and hides the real source.